

PitForge: Exact Ultimate-Pit Optimization Reproducing the Published MineLib Optima, with Whittle Nested Shells and Scheduling

Felipe Santib   ez-Leal 

Technical report · v2.2, 18 September 2026 · Licence CC BY 4.0
 CAOS open-research programme · CAOS_PitForge, open-pit design optimisation
 Version DOI 10.5281/zenodo.22834763 · concept DOI (always latest) 10.5281/zenodo.21519687
 Not peer reviewed. Source code, data and computational records: github.com/fsantibanezleal/CAOS_PitForge

Abstract. The first optimization of open-pit mine design is the ultimate pit limit: given a block model in which each block carries a net economic value and a set of slope-precedence constraints, find the set of blocks to extract that maximises total value while respecting the slopes. The problem has an exact polynomial solution as the maximum-weight closure of the precedence graph, computed as a minimum cut. PitForge implements it in a TypeScript engine that also runs in the browser, where the pit is re-solved as the price, revenue factor or slope change, validates the solver against published optima, and extends it to the two decisions that follow. On the three of MineLib’s eleven instances that have a verified public mirror (newman1, zuck_small and kd, 1060 to 14153 blocks), the solver reproduces the *published* optimum to a relative error of 10^{-9} or less, and an independent pseudoflow solver returns the same values. Beyond the single pit, the tool computes the Whittle nested pit shells, the family of optimal pits over an ascending revenue-factor schedule, with the value, tonnage and strip-ratio curves that guide phase and pushback design, and it solves two separate constrained production-scheduling scenarios. On the published newman1.cpit scenario (six periods, 8% discount and two resources), it reproduces the published 24,486,184 LP bound and finds a feasible 23,553,245 heuristic schedule, a 3.81% gap. A separate synthetic twin (eight periods, 10% discount and one resource) has an 11.29% gap and is not comparable. Every pit value is checked against the flow identity pit value = $\sum$ positive value – max flow on every solve.

Index Terms. Open-pit mining, ultimate pit limit, maximum closure, minimum cut, maximum flow, nested pit shells, Whittle, production scheduling, MineLib.

I. INTRODUCTION

THE ultimate pit limit (UPL) is the foundational optimization of open-pit mine design: given a three-dimensional block model in which each block has a net value (recoverable metal less mining and processing cost) and slope-precedence constraints (a block can be mined only after those above it in the pit wall), which set of blocks maximises total value? The problem has an exact, polynomial-time solution, established by Lerchs and Grossmann [1] and understood through Picard’s

theorem [2] as the maximum-weight closure of the precedence graph, which is the source side of a minimum s – t cut and hence solvable by any maximum-flow algorithm [3], [4]. This is textbook operations research [8]; what a practical tool adds is validation against known answers, the family of nested pits used for phase design, and the connection to the production schedule.

PitForge solves the UPL as a minimum cut and checks the value identity on every solve. It is validated here against the published MineLib benchmark optima [6], and it extends the exact solve to the Whittle nested pit shells [5], the optimal pits across an ascending revenue factor that give the value-tonnage-strip curves phase design rests on, and to a constrained production schedule [7], for which it reports the gap to a certified bound. The contribution is an exact, validated implementation that reproduces the literature and states where exactness ends, not a new algorithm.

II. THE EXACT PIT, VALIDATED ON MINELIB

The ultimate pit is the maximum-weight closed set of the precedence graph, and PitForge computes it by Picard’s reduction to a minimum cut: a source arc to every positive-value block, a sink arc from every negative block, and an infinite-capacity arc for each precedence relation, after which a Dinic maximum flow [9] gives the pit as the source-reachable set. The closure property and the value identity pit value = $\sum_{v_b > 0} v_b$ – maxflow are asserted on every solve, so an inconsistent value cannot be returned without an error. The validation uses three of MineLib’s eleven instances, those with a verified public mirror (Fig. 1, Table I); the other eight, among them marvin and the McLaughlin instances, lack a verified mirror or, for mclaughlin, its precedence file. On all three, PitForge’s value matches the published optimum to a relative error of 9.96×10^{-10} on newman1, 1.86×10^{-10} on zuck_small and 1.30×10^{-10} on kd, which is agreement to floating-point precision, and an independent pseudoflow solver [4] returns the same value on each. Under Node, running the same TypeScript engine the browser runs, the 1060-block newman1 solves in milliseconds and the 14153-block kd in under a second. Those times are machine-dependent, so no decimal figure is quoted; within one run, the pseudoflow solver

F. Santib   ez-Leal is with the CAOS open-research programme, Santiago, Chile (e-mail: fsantibanez@gmail.com; ORCID 0000-0002-0150-3246).

Interactive tool and reproducible pipeline (MIT) at https://github.com/fsantibanezleal/CAOS_PitForge; the tool is available at <https://pitforge.fasl-work.com>.

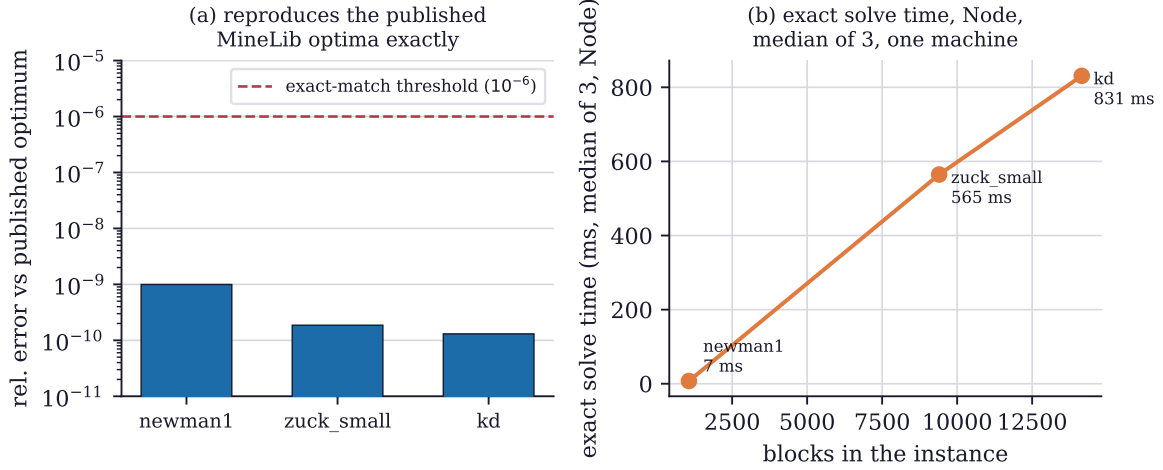


Fig. 1. Validation on the public MineLib instances. (a) The solver reproduces the *published* optimum of newman1, zuck_small and kd to a relative error of 10^{-9} or less. (b) Exact solve time against instance size, measured under Node on one machine as the median of three runs: milliseconds for the 1060-block newman1 and under a second for the 14153-block kd. These are timings of one environment, not a property of the method; repeat runs on the same machine varied by a factor of two to five, and the artifact records the environment beside them. The browser runs the same engine and displays its own measured time; the figure plots the Node measurements.

TABLE I

PITFORGE VERSUS THE PUBLISHED MINELIB OPTIMA. THE SOLVER MATCHES ALL THREE TO A RELATIVE ERROR OF 10^{-9} OR LESS. SOLVE TIMES ARE OMITTED BECAUSE THEY ARE MACHINE-DEPENDENT; THE ARTIFACT RECORDS THEM WITH THE ENVIRONMENT THEY WERE MEASURED IN.

Instance	blocks	published optimum	rel. error
newman1	1060	2.6087×10^7	1.0×10^{-9}
zuck_small	9400	1.4227×10^9	1.9×10^{-10}
kd	14153	6.5220×10^8	1.3×10^{-10}

is one to two orders of magnitude slower than Dinic on the two larger instances.

III. NESTED SHELLS AND SCHEDULING

A mine is not designed as one pit but as a sequence, and PitForge computes the two objects that sequence rests on (Fig. 2). The first is the Whittle nested pit shells [5]: solving the ultimate pit over an ascending schedule of revenue factors (multiplying the metal price) yields a family of nested optimal pits, from the first non-empty pit at a revenue factor of 0.17 to the full pit at a factor of one, and their value, tonnage and strip-ratio curves (Fig. 2a) are what a planner uses to choose phases and pushbacks. Because each shell is an exact ultimate-pit solve, the whole family is exact. The second is the production schedule: assigning the pit's blocks to time periods under capacity constraints to maximise discounted value is the constrained pit scheduling problem [7], harder than the UPL. PitForge bounds it with the linear relaxation of the time-indexed formulation and builds a feasible schedule with an independent greedy heuristic, not a rounding of the relaxation, and reports the gap between the two (Fig. 2b). On the published newman1.cpit definition (six periods, 8% discount, total-movement and processing limits) it reproduces the 24,486,184 bound to a relative error of 3.7×10^{-9} and finds

a feasible 23,553,245 schedule, a 3.81% gap to that bound. MineLib's cited historical feasible value is 23,483,671, a 4.1% gap; the heuristic exceeds it by 0.30%, and no comparison with later published schedules is made here, so no best-known result is claimed. The synthetic-twin scenario has a 104.6 million bound and a 92.8 million feasible schedule, an 11.29% gap. The two scenarios have different denominators, and each gap is stated with its scenario.

IV. DISCUSSION AND SCOPE

The ultimate pit and the nested shells are solved *exactly*, and the reproduction of the MineLib optima to relative error 10^{-9} or less validates the implementation; the schedule is solved to a certified bound with the gap reported, because the scheduling problem, unlike the UPL, does not admit a cheap exact solve at these sizes. The value identity checked on every solve guarantees that a returned pit value is consistent with the computed flow. The validation covers three MineLib instances of up to 14153 blocks; the other eight, all larger (29277 to 2.1 million blocks), are not attempted. MineLib is licensed CC BY-SA 3.0 Unported; source instances are fetched at run time and only attributed summary numbers (the optima and timings) are kept in the repository. The synthetic cases used for the nested shells and the schedule are physically reasonable block models, not a specific deposit, so their numbers illustrate the method rather than predict a mine. The tool computes the pit and the schedule; it does not replace the geological, geotechnical and financial judgement that turns them into a mine plan.

V. CONCLUSION

PitForge solves the open-pit ultimate pit limit exactly, as a maximum-closure minimum cut, and reproduces the three published MineLib optima with a verified public mirror to a relative error of 10^{-9} or less, with an independent pseudoflow

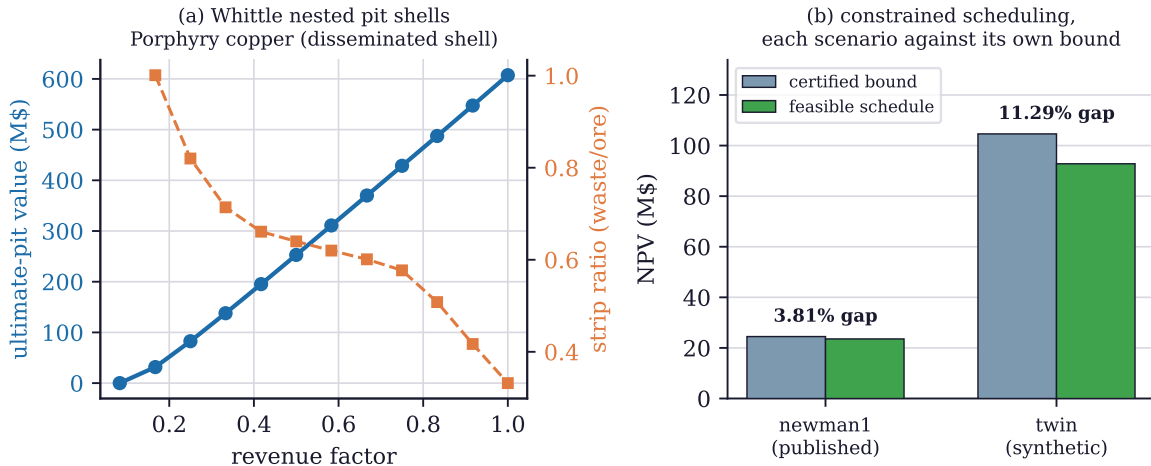


Fig. 2. Beyond the single pit. (a) The Whittle nested pit shells of a porphyry copper case: solving the ultimate pit over an ascending revenue factor gives a family of nested pits whose value rises and whose strip ratio falls as the pit grows. The first shell is empty, since no block pays for itself at that revenue factor, so the value curve starts at zero, and the strip-ratio series, which is waste over ore and therefore undefined without ore, starts at the second shell. (b) The two constrained-scheduling scenarios, each against its own relaxation bound. Left, the published `newman1.cpit` scenario, six periods at 8% discount with two resource constraints: a certified upper bound of 24.49 million against a feasible greedy schedule of 23.55 million, a 3.81% gap, with the bound reproducing MineLib’s published LP bound to 3.7×10^{-9} . Right, a separate synthetic twin, eight periods at 10% discount with one movement limit: 104.6 million against 92.8 million, an 11.29% gap. The two gaps have different denominators and are not one number.

solver agreeing. It extends the exact solve to the Whittle nested pit shells that guide phase design, and to constrained production scheduling, whose gap to a certified bound it reports per scenario: 3.81% for the published `newman1.cpit` scenario and 11.29% for a separate, non-comparable synthetic twin.

APPENDIX A REPRODUCTION

The repository artifacts under `data/derived/` carry the MineLib benchmark (`minelib-results.json`: per-instance published optimum, solved value, relative error, the Dinic and pseudoflow timings and the timing environment), the nested-shell curves per case (`case-results.json`: revenue factor, pit value, tonnage and strip ratio), and the constrained schedules (`cpit-schedule.json`: the certified LP bound and the net present value of the greedy schedule per scenario). The figure script under `manuscripts/ultimate-pit/figures/` reads those artifacts directly and writes the plotted values to `manuscripts/ultimate-pit/data/pf.json`. The exact solver is deterministic, so re-running reproduces Table I and Figs. 1 and 2, except for the machine-dependent timings. The MineLib instances are CC BY-SA 3.0 Unported and are fetched from public mirrors into a local cache that is not kept in the repository.

REFERENCES

- [1] H. Lerchs and I. F. Grossmann, “Optimum design of open-pit mines,” *CIM Bulletin*, vol. 58, pp. 47–54, 1965.
- [2] J.-C. Picard, “Maximal closure of a graph and applications to combinatorial problems,” *Management Science*, vol. 22, no. 11, pp. 1268–1272, 1976. doi:10.1287/mnsc.22.11.1268.
- [3] L. R. Ford and D. R. Fulkerson, “Maximal flow through a network,” *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956. doi:10.4153/CJM-1956-045-5.
- [4] D. S. Hochbaum, “The pseudoflow algorithm: A new algorithm for the maximum-flow problem,” *Operations Research*, vol. 56, no. 4, pp. 992–1009, 2008. doi:10.1287/opre.1080.0524.
- [5] J. Whittle, “Beyond optimization in open pit design,” in *Proc. Canadian Conf. on Computer Applications in the Mineral Industry*, Rotterdam: Balkema, 1988, pp. 331–337.
- [6] D. Espinoza, M. Goycoolea, E. Moreno, and A. Newman, “MineLib: A library of open pit mining problems,” *Annals of Operations Research*, vol. 206, no. 1, pp. 93–114, 2013. doi:10.1007/s10479-012-1258-3.
- [7] R. Chicoisne, D. Espinoza, M. Goycoolea, E. Moreno, and E. Rubio, “A new algorithm for the open-pit mine production scheduling problem,” *Operations Research*, vol. 60, no. 3, pp. 517–528, 2012. doi:10.1287/opre.1120.1050.
- [8] A. M. Newman, E. Rubio, R. Caro, A. Weintraub, and K. Eurek, “A review of operations research in mine planning,” *Interfaces*, vol. 40, no. 3, pp. 222–245, 2010. doi:10.1287/inte.1090.0492.
- [9] E. A. Dinic, “Algorithm for solution of a problem of maximum flow in networks with power estimation,” *Soviet Mathematics Doklady*, vol. 11, pp. 1277–1280, 1970.